USE CASE SPECIFICATIONS

ZPIN E-Commerce Platform

Project: zpinecom

Version: 1.0

Date: May 2026

Author: Business Analyst / Product Owner

#	Epic Name
1	Authentication & Onboarding
2	Product Discovery
3	Cart & Wishlist
4	Checkout
5	Delivery & Logistics
6	ZPIN Wallet & Payments
7	Order Lifecycle
8	Product Reviews & Ratings
9	Admin Operations
10	Notifications

1 AUTHENTICATION & ONBOARDING

1.1 Brief Description

This use case covers all activities related to user registration, email verification, login, session management, password reset, and logout on the ZPIN e-commerce platform. It ensures that only verified and authenticated customers can access their accounts and perform shopping activities.

1.2 Actors

- Customer — the primary user who registers, logs in, and manages their account
- System — handles OTP generation, token management, and session validation

1.3 Flow of Events

1.3.1 Basic Flow

1.3.1.1 User Registration

1) The customer navigates to the Sign Up screen.

2) The system requests the customer to enter their name, email address, mobile number, and a password (minimum 8 characters containing at least one letter and one number).

3) The customer submits the registration form.

4) The system validates the input, creates the account, sends a 6-digit OTP to the registered email, and navigates the customer to the OTP Verification screen.

1.3.1.2 Email OTP Verification

1) The system presents the OTP Verification screen.

2) The customer enters the 6-digit OTP received in their email.

3) The system validates the OTP, marks the account as verified, issues an access token (15-minute TTL) and a refresh token (30-day TTL), stores them in AsyncStorage, and redirects the customer to the Home screen.

1.3.1.3 User Login

1) The customer navigates to the Login screen.

2) The system requests the customer to enter their email and password.

3) The customer submits the credentials.

4) The system validates the credentials, issues access and refresh tokens, and redirects the customer to the Home screen.

1.3.1.4 Persistent Session and Token Refresh

1) When the customer reopens the app, the system silently checks for a stored refresh token.

2) If a valid refresh token exists, the system exchanges it for a new access token without requiring the customer to log in again.

3) If an API call is made mid-session while the access token has expired, the Axios interceptor automatically uses the refresh token to obtain a new access token and retries the original request.

4) On each successful token exchange, the old refresh token is deleted and a new one is issued (token rotation).

1.3.1.5 Forgot Password / Password Reset

1) The customer navigates to the Forgot Password screen and enters their registered email address.

2) The system sends a 6-digit OTP to the email and navigates the customer to the Verify Reset OTP screen.

3) The customer enters the correct OTP and receives a signed single-use reset token.

4) The customer enters a new password meeting strength requirements. The system updates the password, invalidates the reset token, and redirects the customer to the Login screen.

1.3.1.6 Logout

1) The customer taps Logout in their profile.

2) The system calls the logout API to invalidate the refresh token in the database.

3) The system removes the access token and refresh token from AsyncStorage and redirects the customer to the Login screen.

1.3.2 Alternative Flows

1.3.2.1 Duplicate Email or Mobile on Registration

If the customer enters an email or mobile number already registered, the system displays a generic error message without revealing whether the specific account exists, to prevent user enumeration.

1.3.2.2 Weak Password

If the customer enters a password shorter than 8 characters or one without the required letter and number combination, the system displays an inline validation error and does not submit the form.

1.3.2.3 Registration Rate Limit Exceeded

If the customer submits the registration form 4 or more times within one hour from the same IP address, the system returns a rate-limit error (429) and asks the customer to try again later.

1.3.2.4 Incorrect OTP

If the customer enters an incorrect OTP, the system displays an error and clears the OTP field for re-entry.

1.3.2.5 Expired OTP

If the OTP has expired, the system displays an OTP expired message and provides a Resend OTP option. If the customer requests more than 5 resends within 15 minutes, a rate-limit error is shown.

1.3.2.6 Invalid Login Credentials

If the customer enters an incorrect password or a non-existent email, the system shows a generic Invalid credentials error (same message in both cases) to prevent user enumeration.

1.3.2.7 Login Rate Limit Exceeded

If the customer fails login 10 times within 15 minutes from the same IP, subsequent attempts are blocked with a 429 rate-limit error for the remainder of the window.

1.3.2.8 Unverified Account Login

If the customer attempts to log in with an account where is_verified is false, the system redirects the customer to the OTP Verification screen instead of the Home screen.

1.3.2.9 Deactivated Account Login

If the customer's account is deactivated (is_active = false), the system displays an Account disabled error.

1.3.2.10 Session Expiry

If both the access token and the refresh token have expired or been invalidated, the system logs the customer out and redirects them to the Login screen with a Session expired message.

1.3.2.11 Expired or Used Reset Token

If the customer attempts to use an already-used or expired reset token, the system returns an Invalid or expired token error and does not update the password.

1.3.2.12 Logout with Network Failure

If the logout API call fails due to a network error, the system still clears the tokens from AsyncStorage locally and redirects the customer to the Login screen.

1.4 Special Requirements

- Access token TTL: 15 minutes; Refresh token TTL: 30 days.
- Token rotation must be implemented: each refresh token is single-use and a new one is issued on every exchange.
- All tokens must be stored in AsyncStorage on the mobile client.
- OTP resend rate limit: maximum 5 resends within 15 minutes.
- Registration rate limit: maximum 3 attempts per hour per IP.
- Login rate limit: 10 failed attempts per 15-minute window per IP results in lockout.
- User enumeration prevention must be applied across registration, login, and forgot password flows.

1.5 Preconditions

- The email OTP service (mailer) must be operational before registration or password reset begins.
- For login, the customer must have a registered account in the system.
- For token refresh, the customer must have a valid non-expired refresh token stored on the device.

1.6 Postconditions

- After successful registration and OTP verification, the customer account is active and the customer is logged in with valid tokens.
- After successful login, the customer is redirected to the Home screen with active session tokens.
- After password reset, the old reset token is invalidated and the customer is redirected to the Login screen.
- After logout, all session tokens are invalidated on the server and cleared from the device.

2 PRODUCT DISCOVERY

2.1 Brief Description

This use case covers how a customer discovers products on the ZPIN platform. It includes viewing the home feed with personalized recommendations and fresh categories, browsing products by category with filtering options, searching for products by keyword, viewing full product details, and receiving personalized product recommendations.

2.2 Actors

- Customer — browses, searches, and views product details
- System — fetches product data, applies recommendation logic, and handles offline fallback

2.3 Flow of Events

2.3.1 Basic Flow

2.3.1.1 Home Screen Feed

1) The customer opens the Home screen.

2) The system fetches personalized recommended products from the recommendations endpoint.

3) The system also displays a Fresh Categories section showing the top 5 categories with the most recently added products.

4) The customer taps a product card and is navigated to the Product Detail screen.

2.3.1.2 Category Browse

1) The customer navigates to the Categories tab.

2) The system displays the full category tree (Men, Women, Kids, Makeup) with live product counts.

3) The customer taps a category. The system navigates to the Product Listing screen and fetches all products in that category.

4) The customer can open a filter modal to sort by price (low to high, high to low) and filter by price range, brand, rating, and discount percentage.

5) The product grid updates immediately using client-side filtering without a new API call.

2.3.1.3 Product Search

1) The customer types a search term in the search bar.

2) After the customer pauses typing (debounce), the system sends a search request with the query term.

3) The system returns matching products, applying normalized text matching (e.g., keychain matches key chain).

4) The customer taps a search result and is navigated to the Product Detail screen.

2.3.1.4 Product Detail Page

1) The customer navigates to a product detail page.

2) The system displays a scrollable image gallery, product name, current price, average rating, stock status, size and color selectors, and a product description.

3) The customer selects a size or color variant and taps Add to Cart.

4) The customer can scroll to the reviews section to see paginated reviews with ratings, comments, review images, and Verified Purchase badges.

2.3.1.5 Personalized Recommendations

1) The system computes a recommendation profile for the customer based on activity weights:
   – Purchased products: +3 weight
   – Wishlisted products: +2 weight
   – Search terms: +1 weight
2) Products already purchased by the customer are deprioritized or excluded from recommendations.
3) The recommendations are displayed on the Home screen.

2.3.2 Alternative Flows

2.3.2.1 Backend Unreachable

If the backend is unreachable when the customer opens the Home screen, the system displays the static fallback product list from the local constants file and shows a subtle offline indicator.

2.3.2.2 New Customer with No History

If the customer has no purchase, wishlist, or search history, the recommendations section falls back to displaying the latest products available on the platform.

2.3.2.3 Category with No Products

If a category has zero products, the system displays an empty state illustration with a message indicating that no products are available in that category.

2.3.2.4 No Search Results

If the search query returns no matching products, the system displays an empty state with a message indicating no results were found and suggesting the customer try different keywords.

2.3.2.5 Product Out of Stock

If a product is out of stock, the Add to Cart button is replaced with an Out of Stock indicator and cannot be tapped.

2.3.2.6 Add to Cart Without Size Selection

If the customer taps Add to Cart without selecting a required size, the system displays an inline prompt asking the customer to select a size before proceeding.

2.4 Special Requirements

- Search input must be debounced to avoid excessive API calls on every keystroke.
- Category filtering must be performed client-side on the already-fetched product list, without a new API call.
- A static fallback product list must be maintained locally for offline scenarios.
- Size variants with zero stock must be shown as disabled or greyed out on the Product Detail screen.

2.5 Preconditions

- The customer must be logged into the system.
- For personalized recommendations, the system must have access to the customer's activity history.

2.6 Postconditions

• After browsing and selecting a product, the customer is on the Product Detail screen ready to add the item to the cart or wishlist.

• Recommendation scores are updated as the customer interacts with products.

3 CART & WISHLIST

3.1 Brief Description

This use case allows the customer to manage their shopping cart and wishlist. It includes adding products to the cart, viewing and updating cart items, clearing the cart, saving products to the wishlist, and moving wishlist items directly to the cart. The system validates stock levels and prices in real time.

3.2 Actors

- Customer — adds, removes, and manages cart and wishlist items
- System — validates stock levels and prices, updates cart totals

3.3 Flow of Events

3.3.1 Basic Flow

3.3.1.1 Add to Cart

1) The customer selects a product variant (size and color) on the Product Detail screen and taps Add to Cart.

2) The system sends an add-to-cart request and increments the cart badge count.

3) If the product is already in the cart, the system auto-increments the quantity instead of creating a duplicate entry.

4) The system captures the price at the time of adding (price_at_add) for each cart item.

3.3.1.2 View and Manage Cart

1) The customer opens the Cart screen.

2) The system displays all cart items with their name, image, selected variant, quantity, and price, along with a price breakdown including subtotal, shipping estimate, and tax.

3) The customer can increase or decrease item quantities, or remove items from the cart.

4) The price breakdown updates immediately after any change.

3.3.1.3 Clear Cart

1) The customer taps Clear Cart.

2) The system displays a confirmation dialog.

3) The customer confirms, and the system removes all items from the cart and shows an empty state.

3.3.1.4 Add to Wishlist

1) The customer taps the wishlist icon on a product card or product detail page.

2) The system adds the product to the wishlist and changes the icon to a filled active state.

3) If the icon is tapped again, the product is removed from the wishlist (toggle behavior).

3.3.1.5 Move Wishlist Item to Cart

1) The customer opens the Wishlist screen and taps Add to Cart on a wishlist item.

2) The system adds the item to the cart and displays a success toast.

3.3.2 Alternative Flows

3.3.2.1 Quantity Exceeds Available Stock on Add

If the requested quantity exceeds available stock when adding to cart, the system displays an error message indicating how many items are available, and the cart is not updated.

3.3.2.2 Product Goes Out of Stock During Add

If the product goes out of stock between the page load and the add-to-cart action, the system returns a stock validation error and refreshes the Product Detail screen to show the updated stock status.

3.3.2.3 Price Change Detected on Cart Load

If a product's price has changed since the customer added it to the cart, the system flags that item with a Price updated notice showing the new price.

3.3.2.4 Out-of-Stock Item in Cart

If a cart item's product is now out of stock, the system flags the item with an Out of Stock warning and disables the Proceed to Checkout button until the item is removed.

3.3.2.5 Quantity Increase Beyond Stock on Update

If the customer tries to increase an item's quantity beyond available stock, the quantity is capped at the available stock level and the customer sees a warning message.

3.3.2.6 Clear Cart Cancelled

If the customer cancels the clear cart confirmation dialog, no items are removed and the cart remains unchanged.

3.3.2.7 Duplicate Wishlist Add

If the customer tries to add a product already in the wishlist via the API, the backend returns a 409 conflict. The frontend handles this gracefully without showing an error to the customer.

3.3.2.8 Wishlist Item Requires Size Selection

If a wishlist item requires a size selection and none is stored, tapping Add to Cart navigates the customer to the Product Detail screen to select a size before adding to the cart.

3.3.2.9 Wishlisted Item Out of Stock

If a wishlisted product goes out of stock, the item is flagged with an Out of Stock badge on the Wishlist screen and the Add to Cart button is disabled.

3.4 Special Requirements

- Price and stock validation must be performed server-side in real time when the cart is loaded.
- The price at the time of adding (price_at_add) must be captured and stored per cart item.
- A confirmation dialog is required before clearing the entire cart.

3.5 Preconditions

- The customer must be logged into the system.
- For adding to cart, the customer must select all required product variants (size and color) on the Product Detail screen.

3.6 Postconditions

- After a successful add-to-cart, the item appears in the cart with the correct quantity and price snapshot, and the cart badge count is updated.
- After clearing the cart, the cart is empty and shows the empty state.

• After adding to the wishlist, the product is saved and the wishlist icon is updated to reflect the saved state.

4 CHECKOUT

4.1 Brief Description

This use case describes the multi-step checkout process on the ZPIN platform. It covers address selection, payment method selection, coupon application, browsing available coupons, and reviewing and placing the final order. The process uses a shared CheckoutContext to pass data between steps.

4.2 Actors

- Customer — selects address, payment method, applies coupons, and places the order
- System — validates stock, wallet balance, coupon eligibility, and processes order placement

4.3 Flow of Events

4.3.1 Basic Flow

4.3.1.1 Step 1 — Address Selection

1) The customer enters the checkout flow.

2) The system displays a list of saved shipping addresses with labels (Home, Work, Other) and pre-selects the default address.

3) The customer selects an address and taps Continue. The selected address is stored in CheckoutContext and the customer is navigated to the Payment screen.

4) Alternatively, the customer taps Add New Address and fills in address details (address line, city, state, pincode, phone, label). The address is encrypted and saved to the database.

4.3.1.2 Step 2 — Payment Method Selection

1) The system displays available payment options: Cash on Delivery (COD), ZPIN Wallet with the live balance displayed, and Razorpay (card, UPI, netbanking).

2) The customer selects their preferred payment method and proceeds.

4.3.1.3 Coupon Application

1) The customer enters a coupon code on the Payment screen and taps Apply.

2) The system validates the coupon and, if valid, calculates the discount and updates the price breakdown.

3) For percentage discounts with a maximum cap, the system applies the lesser of the calculated percentage and the cap.

4.3.1.4 Browse Available Coupons

1) The customer navigates to the Coupons screen.

2) The system displays all coupons with their code, discount description, minimum order amount, expiry date, and status.

3) The customer can copy a coupon code to the clipboard or tap Apply to be returned to the payment screen with the coupon pre-filled.

4.3.1.5 Step 3 — Order Summary and Place Order

1) The customer reviews the order summary showing all items, the shipping address, the payment method, the applied coupon (if any), and the full price breakdown.

2) The customer taps Place Order. The system sends an order creation request with all checkout context data.

3) On successful order creation, the cart is cleared, the customer receives an order confirmation notification, and is navigated to the Order Confirmation screen.

4.3.2 Alternative Flows

4.3.2.1 No Saved Addresses

If the customer has no saved addresses when entering checkout, the system prompts them to add a new address before they can proceed.

4.3.2.2 Invalid Pincode

If the customer enters a pincode that is not 6 digits, the system displays an inline validation error and the form cannot be submitted.

4.3.2.3 Insufficient Wallet Balance

If the customer selects ZPIN Wallet and the balance is less than the order total, the system displays the balance, the order total, and the shortfall amount. The Continue button is disabled.

4.3.2.4 Expired Coupon

If the customer enters an expired coupon code, the system displays an error message indicating the coupon has expired.

4.3.2.5 Coupon Below Minimum Order Amount

If the order total is below the coupon's minimum order amount, the system displays an error showing the required minimum order value.

4.3.2.6 Coupon Usage Limit Reached

If the coupon's usage limit has been reached, the system displays an error message indicating the coupon is no longer available.

4.3.2.7 Invalid Coupon Code

If the customer enters a coupon code that does not exist in the system, an Invalid coupon code error is shown.

4.3.2.8 Coupon Auto-Invalidated After Cart Change

If a valid coupon is applied and the customer removes a cart item causing the total to drop below the minimum order amount, the coupon is automatically invalidated and the discount is removed.

4.3.2.9 Out-of-Stock Item Detected at Order Placement

If a cart item goes out of stock between the summary screen load and the Place Order action, the order creation fails with a stock error, the item is flagged in the cart, and the customer is redirected back to the cart to resolve the issue.

4.3.2.10 Wallet Race Condition at Order Placement

If the customer's wallet balance drops below the required amount between the payment screen and order placement, the order creation fails with an Insufficient wallet balance error and the customer is returned to the payment screen.

4.3.2.11 Expired Coupon Shown on Browse Screen

Expired coupons are shown on the Browse Coupons screen with an Expired badge and cannot be applied.

4.4 Special Requirements

• Shipping addresses must be encrypted at rest in the database.
• CheckoutContext must persist all selected data (address, payment method, coupon) across steps.
• Coupon validation, stock checks, and wallet balance checks must be re-performed server-side at the time of order placement to handle race conditions.
• For percentage coupons with a cap, the discount must not exceed the specified maximum amount.

4.5 Preconditions

• The customer must be logged into the system.
• The customer's cart must contain at least one item that is in stock.
• For wallet payment, the customer must have an active ZPIN Wallet.

4.6 Postconditions

• After successful order placement, the cart is cleared, an order confirmation notification is sent, and an order record is created in the system.
• If the order placement fails, the system state is unchanged and the customer is informed of the reason for failure.

5 DELIVERY & LOGISTICS

5.1 Brief Description

This use case covers the selection of a delivery method during checkout and the tracking of order delivery. It includes Standard Delivery (3 to 7 business days) and Instant Trial Delivery (faster delivery with a non-refundable trial fee). It also covers how the customer tracks their order progress through a delivery timeline.

5.2 Actors

- Customer — selects delivery method and tracks order
- System — calculates delivery fees, sets delivery timelines, and updates tracking status

5.3 Flow of Events

5.3.1 Basic Flow

5.3.1.1 Select Standard Delivery

1) The customer is in the checkout flow and views the delivery options.

2) Standard Delivery is shown as the default option.

3) The system calculates the shipping fee: free if the order subtotal is above Rs 500, otherwise Rs 49.

4) The customer selects Standard Delivery. When the order is placed, the estimated delivery date is set to 3 to 7 business days from the order date.

5) On the tracking screen, a STD badge is displayed and the Standard Delivery tracking card is shown.

5.3.1.2 Select Instant Trial Delivery

1) The customer views the delivery options and sees Instant Trial Delivery alongside Standard Delivery.

2) The system displays a clear description of the trial concept and the Rs 50 non-refundable trial fee.

3) The customer selects Instant Trial Delivery. The system adds the Rs 50 trial fee to the price breakdown and updates the final amount.

4) When the order is placed, the order notes field is set to instant_delivery and the estimated delivery date reflects the faster timeline.

5) On the tracking screen, a FAST badge is displayed and the Instant Trial tracking card is shown with trial-specific instructions.

5.3.1.3 Track Order Delivery

1) The customer navigates to the order tracking screen.

2) The system displays an order status timeline showing the stages from order placed to delivered.

3) The current active stage is highlighted and the estimated delivery date is prominently displayed.

4) When the order status reaches delivered, all timeline steps are shown as completed and a Return Order button appears.

5.3.2 Alternative Flows

5.3.2.1 Instant Trial Cancellation Refund

If the customer cancels an Instant Trial Delivery order that is still in pending or confirmed status, the system processes the refund but excludes the Rs 50 non-refundable trial fee from the refund amount.

5.3.2.2 Tracking API Failure

If the tracking API call fails due to a network error, the system displays a message indicating that tracking information could not be loaded, along with a retry button.

5.4 Special Requirements

- Standard Delivery: free for orders above Rs 500, Rs 49 for orders at or below Rs 500.
- Instant Trial Delivery: a Rs 50 non-refundable trial fee is added to the order total.
- The order notes field must be set to instant_delivery for Instant Trial orders to distinguish them from Standard orders.
- The Rs 50 trial fee must be excluded from any refund calculation for Instant Trial orders.

5.5 Preconditions

- The customer must be in the checkout flow with a valid cart and a selected shipping address.
- For order tracking, the customer must have an existing order in the system.

5.6 Postconditions

- After selecting a delivery method, the delivery fee and estimated delivery date are reflected in the order summary.
- After the order is delivered, the tracking screen shows all steps as completed and the return option becomes available.

6 ZPIN WALLET & PAYMENTS

6.1 Brief Description

This use case covers all activities related to the ZPIN Wallet and payment processing. It includes viewing the wallet balance and transaction history, paying for orders using the wallet, paying via the Razorpay payment gateway (card, UPI, netbanking), and receiving wallet credits as refunds upon order cancellation or approved return requests.

6.2 Actors

- Customer — views wallet, uses wallet for payment, and receives refund credits
- Admin — triggers refund credits by approving return requests
- System — manages wallet transactions atomically and interfaces with the Razorpay payment gateway

6.3 Flow of Events

6.3.1 Basic Flow

6.3.1.1 View Wallet Balance and Transaction History

1) The customer navigates to the Payment Methods screen.

2) The system displays the current ZPIN Wallet balance (decrypted and shown in INR).

3) The customer scrolls to the transaction history section. The system displays the last 20 transactions, each showing the type (credit or debit), amount, description, and date.

4) Credit transactions (such as refunds) are shown in green with a plus prefix. Debit transactions (such as order payments) are shown in red with a minus prefix.

6.3.1.2 Pay with ZPIN Wallet at Checkout

1) The customer selects ZPIN Wallet as the payment method and the balance is sufficient to cover the order total.

2) The customer places the order. The system debits the wallet atomically within the same database transaction as the order creation.

3) A wallet transaction record is created with type debit, the order amount, and a reference to the order ID.

6.3.1.3 Pay with Razorpay

1) The customer selects Razorpay and taps Place Order.

2) The system creates a Razorpay order and launches the Razorpay SDK payment sheet.

3) The customer completes the payment in the Razorpay sheet.

4) The frontend sends the payment ID, order ID, and signature to the backend for verification.

5) The backend verifies the HMAC-SHA256 signature and, if valid, marks the order as paid and confirmed.

6) The customer receives a payment success notification and is navigated to the Order Confirmation screen.

6.3.1.4 Wallet Credit on Order Cancellation

1) The customer cancels an order and selects Refund to ZPIN Wallet as the refund method.

2) The system credits the wallet with the refund amount (excluding any non-refundable fees such as the Instant Trial fee).

3) A wallet transaction record is created with type credit, the refund amount, and a reference to the cancelled order ID.

4) The updated balance is immediately reflected when the customer views the wallet.

6.3.1.5 Wallet Credit on Return Refund

1) The admin updates a return request status to refunded.

2) The system automatically credits the customer's wallet with the refund amount for the returned items.

3) If the order was an Instant Trial Delivery, the Rs 50 trial fee is excluded from the refund amount.

4) A wallet transaction record is created and the order's payment_status is updated to refunded.

6.3.2 Alternative Flows

6.3.2.1 New Customer Wallet Auto-Creation

If the customer has no wallet and accesses the Payment Methods screen for the first time, the system automatically creates a wallet with a Rs 0 balance.

6.3.2.2 Insufficient Wallet Balance

If the customer selects ZPIN Wallet and the balance is less than the order total, the system displays the balance, the order total, and the shortfall amount. The Place Order button is disabled.

6.3.2.3 Wallet Race Condition

If two concurrent requests attempt to debit the wallet simultaneously, the database transaction ensures only one debit succeeds. The other fails gracefully with an Insufficient balance error.

6.3.2.4 Wallet Debit Rollback on Order Failure

If the order creation fails after the wallet has been debited (for example, due to a stock error), the wallet debit is rolled back within the same database transaction, leaving the balance unchanged.

6.3.2.5 Razorpay Payment Failure or Dismissal

If the Razorpay payment fails or is dismissed by the customer, the order remains in pending status and the customer is shown an error with an option to retry the payment.

6.3.2.6 Razorpay Signature Verification Failure

If the HMAC-SHA256 signature verification fails on the backend, the order is not marked as paid and the customer sees a payment verification failed message.

6.3.2.7 COD Cancellation Refund to Wallet

If the original payment was COD and the customer selects wallet refund upon cancellation, the system credits the wallet with the order's final amount excluding any non-refundable fees.

6.4 Special Requirements

- Wallet balance must be stored encrypted in the database.
- All wallet debit operations during order creation must be performed within an atomic database transaction to prevent race conditions.
- Razorpay payment verification must use HMAC-SHA256 signature validation on the backend.
- The Rs 50 Instant Trial fee must always be excluded from any refund calculation.

6.5 Preconditions

- The customer must be logged into the system.

- For wallet payment, the customer must have a ZPIN Wallet with sufficient balance.
- For return refund credits, the admin must have approved the return and set the status to refunded.

6.6 Postconditions

- After a successful wallet payment, the wallet is debited and a transaction record is created.
- After a successful Razorpay payment, the order is marked as paid and confirmed.
- After a refund credit, the wallet balance is updated and the transaction appears in the history.

7 ORDER LIFECYCLE

7.1 Brief Description

This use case describes the full lifecycle of an order on the ZPIN platform. It covers placing an order, viewing order history and details, tracking delivery status, cancelling an order, submitting a return request, and tracking the return status.

7.2 Actors

- Customer — places, views, cancels, and returns orders
- Admin — updates order status and manages return workflows
- System — creates orders, applies GST, manages status transitions, and sends notifications

7.3 Flow of Events

7.3.1 Basic Flow

7.3.1.1 Place an Order

1) The customer completes all checkout steps and taps Place Order.

2) The system creates an order record with a human-readable order number (format: ZPIN-XXXXX), calculates the final amount (subtotal plus shipping plus 18 percent GST minus coupon discount), and clears the cart.

3) The customer receives an in-app order confirmation notification and is shown an Order Confirmation screen with the order number and estimated delivery date.

4) If the order involves multiple sellers, each relevant seller receives a new order notification.

7.3.1.2 View Order History

1) The customer navigates to the Orders screen.

2) The system displays a paginated list of orders sorted by most recent first, each showing the order number, date, status, and total amount.

3) The customer can scroll down to load more orders via infinite scroll or a Load More button.

4) The customer taps an order and is navigated to the Order Details screen.

7.3.1.3 View Order Details

1) The system displays all ordered items with their name, image, variant, quantity, and price snapshot at the time of order.

2) The full price breakdown is shown: subtotal, shipping, tax, coupon discount (if applied), and final amount.

3) The decrypted shipping address and the payment method used are displayed.

4) A Cancel Order button is visible if the order is in pending or confirmed status. A Return Order button is visible if the order is in delivered status.

7.3.1.4 Cancel an Order

1) The customer taps Cancel Order on an order in pending or confirmed status.

2) The system displays a cancellation reason selection screen.

3) The customer selects a reason and confirms. The system updates the order status to cancelled.

4) If the order was paid via wallet, the system automatically credits the wallet with the full refund amount.

5) If the order was paid via Razorpay or COD, the customer selects a refund method (original payment method or ZPIN Wallet) and the refund is processed accordingly.

7.3.1.5 Submit a Return Request

1) The customer views a delivered order and taps Return Order.

2) The system displays a screen to select which items to return and enter a reason.

3) The customer submits the return request. The system creates a return record with status requested.

4) The customer sees a confirmation message and can track the return status.

7.3.1.6 Track Return Status

1) The customer views an active return request.

2) The system displays the current return status with a timeline of all status changes and their timestamps.

3) When the admin updates the status to picked_up, the customer sees an Item Picked Up milestone.

4) When the admin updates the status to refunded, the customer sees a Refund Processed milestone and the refund amount is credited to their wallet.

7.3.2 Alternative Flows

7.3.2.1 Order Rate Limit Exceeded

If the customer attempts to place more than 10 orders within one hour, subsequent attempts are blocked with a rate-limit error.

7.3.2.2 No Orders in History

If the customer has no past orders, the Orders screen displays an empty state with a message encouraging the customer to start shopping.

7.3.2.3 Cancel Order Not Allowed

If the order is in shipped or delivered status, the Cancel Order button is not shown. The customer must use the return flow instead.

7.3.2.4 Cancellation of Already-Cancelled Order

If the customer attempts to cancel an order that is already cancelled via the API, the system returns a 400 error with a descriptive message.

7.3.2.5 Return on Non-Delivered Order

If the customer attempts to submit a return request for an order that is not in delivered status, the system returns an error and the return option is not shown.

7.4 Special Requirements

- Order number format: ZPIN-XXXXX (human-readable, unique).
- 18 percent GST must be applied to all orders at the time of creation.
- Order placement rate limit: maximum 10 orders per hour per customer.
- Return requests can only be submitted for orders in delivered status.
- Return status transitions are one-directional: requested to picked_up to refunded. Backward transitions are not allowed.

7.5 Preconditions

- The customer must be logged into the system.
- For order cancellation, the order must be in pending or confirmed status.
- For return submission, the order must be in delivered status.

7.6 Postconditions

- After a successful order placement, an order record is created, the cart is cleared, and notifications are sent.
- After cancellation, the order status is updated to cancelled and the appropriate refund is initiated.
- After a return is approved and refunded, the customer's wallet is credited and the order payment_status is updated to refunded.

8 PRODUCT REVIEWS & RATINGS

8.1 Brief Description

This use case allows customers to submit, edit, delete, and read product reviews and ratings. Reviews include a star rating (1 to 5), an optional comment, and optional images. Customers who have purchased the product receive a Verified Purchase badge on their review. The product's average rating is recalculated automatically when a review is added, updated, or deleted.

8.2 Actors

- Customer — submits, edits, deletes, and reads product reviews
- System — verifies purchase status, enforces one-review-per-product constraint, and recalculates average ratings

8.3 Flow of Events

8.3.1 Basic Flow

8.3.1.1 Submit a Product Review

1) The customer navigates to a product's reviews section and taps Write a Review.

2) The customer submits a star rating (1 to 5), an optional written comment, and optional images.

3) The system saves the review. If the customer has a delivered order containing this product, the review is automatically marked with is_verified_purchase = true and displays a Verified Purchase badge.

4) If the customer has not purchased the product, the review is saved with is_verified_purchase = false and no badge is shown.

8.3.1.2 Edit a Product Review

1) The customer views their own review on a product page.

2) The customer taps Edit and modifies the rating, comment, or images.

3) The system updates the review.

8.3.1.3 Delete a Product Review

1) The customer taps Delete on their own review and confirms the action.

2) The system removes the review and recalculates the product's average rating to reflect the removal.

8.3.1.4 View Product Reviews

1) The customer scrolls to the reviews section on a product detail page.

2) The system displays the average rating (out of 5), the total review count, and a paginated list of individual reviews.

3) Reviews with images display a scrollable thumbnail row. Reviews with is_verified_purchase = true display a Verified Purchase badge.

4) The customer can scroll to the bottom to load the next page of reviews.

8.3.2 Alternative Flows

8.3.2.1 Customer Has Already Reviewed the Product

If the customer has already submitted a review for this product, the Write a Review button is replaced with an Edit Review button. This is enforced by a unique constraint on the combination of user ID and product ID in the database.

8.3.2.2 Unauthorized Edit or Delete

If a customer attempts to edit or delete another user's review via the API, the system returns a 403 Forbidden error and the action is not performed.

8.3.2.3 No Reviews Yet

If a product has no reviews, the reviews section displays an empty state encouraging the customer to be the first to submit a review.

8.4 Special Requirements

- A unique constraint must be enforced on the combination of user ID and product ID to prevent duplicate reviews.
- Verified purchase status (is_verified_purchase) must be determined automatically by the system by checking whether the customer has a delivered order containing the reviewed product.
- The product's average rating must be recalculated whenever a review is created, updated, or deleted.
- Reviews are paginated to prevent performance issues when a product has a large number of reviews.

8.5 Preconditions

- The customer must be logged into the system.
- For editing or deleting a review, the review must belong to the currently logged-in customer.

8.6 Postconditions

- After submitting a review, the review appears on the product page and the product's average rating is updated.
- After deleting a review, the review is removed and the product's average rating is recalculated.
- After editing a review, the updated review is reflected on the product page.

9 ADMIN OPERATIONS

9.1 Brief Description

This use case covers the administrative functions performed by the Admin actor on the ZPIN platform. It includes confirming orders, updating order status through the full lifecycle, managing return requests (pickup and refund), creating and managing discount coupons, and viewing seller notifications for new orders.

9.2 Actors

- Admin — manages order statuses, return workflows, and coupon CRUD operations
- Seller — receives notifications for new orders containing their products
- Customer — is notified of status changes and receives refunds
- System — enforces valid status transitions and manages wallet credits

9.3 Flow of Events

9.3.1 Basic Flow

9.3.1.1 Confirm an Order

1) An order is in pending status.

2) The admin calls the confirm order endpoint.

3) The system updates the order status to confirmed and sends an in-app notification to the customer confirming their order.

4) If the order is an Instant Trial Delivery, the system flags the order appropriately so the customer sees the FAST badge on the tracking screen.

9.3.1.2 Update Order Status Through Lifecycle

1) The admin updates the order through the following valid status transitions:

– confirmed to processing

– processing to shipped (with a tracking number)

– shipped to delivered

2) When marking an order as shipped, the admin provides a tracking number. The system stores it and displays it on the customer's tracking screen.

9.3.1.3 Manage Return Requests

1) A return request is in requested status.

2) The admin updates the return status to picked_up. The system records the picked_up_at timestamp.

3) The admin then updates the return status to refunded with a refund method. The system credits the customer's wallet, records the refunded_at timestamp, and updates the order's payment_status to refunded.

4) For Instant Trial Delivery orders, the Rs 50 trial fee is excluded from the refund amount.

5) Every status change is recorded in the status history JSONB array with the new status and timestamp.

9.3.1.4 Create a Coupon

1) The admin creates a coupon by specifying: a unique code, discount type (percentage or fixed), discount value, minimum order amount, optional maximum discount cap, optional expiry date, and

optional usage limit.

2) The system saves the coupon and makes it available to customers.

9.3.1.5 Manage Existing Coupons

1) The admin can deactivate a coupon by setting is_active = false. The coupon is removed from the active coupons list and cannot be applied.

2) When a coupon's usage count reaches its usage limit, the system automatically makes it invalid.

9.3.1.6 View Seller Notifications

1) When a new order is placed containing products from a seller's account, the system creates a new_order notification with the full order details and the decrypted shipping address.

2) The seller fetches notifications from the notifications endpoint and sees all unread new_order notifications.

3) Notifications are automatically marked as read when fetched.

4) If an order contains products from multiple sellers, each seller receives their own separate notification.

9.3.2 Alternative Flows

9.3.2.1 Confirm Order in Invalid Status

If the admin attempts to confirm an order that is not in pending status, the system returns a 400 error with a descriptive message.

9.3.2.2 Invalid Status Transition

If the admin attempts to skip a step in the order status lifecycle (for example, moving directly from pending to shipped), the system enforces the valid transition sequence and returns an error.

9.3.2.3 Reverse Return Status Transition

If the admin attempts to move a return status backwards (for example, from refunded back to requested), the system returns an error and does not allow the backward transition.

9.3.2.4 Coupon with Past Expiry Date

If the admin creates a coupon with an expiry date in the past, the coupon is immediately invalid and not shown to customers.

9.4 Special Requirements

- Order status transitions must be strictly enforced in the sequence: pending, confirmed, processing, shipped, delivered.
- Return status transitions must follow the sequence: requested, picked_up, refunded. Backward transitions are not permitted.
- Coupon discount types supported: percentage and fixed amount. Percentage coupons may have an optional maximum discount cap.
- The Rs 50 Instant Trial fee must always be excluded from return refund calculations.
- All return status changes must be recorded in a JSONB status history array with timestamps.

9.5 Preconditions

- The admin must be authenticated and have the Administrator role in the system.
- For order confirmation, the order must be in pending status.
- For return management, a return request must already exist in the system.

9.6 Postconditions

- After order confirmation, the order status is updated to confirmed and the customer is notified.
- After return refund processing, the customer's wallet is credited and the order payment_status is updated to refunded.
- After coupon creation, the coupon is available for customers to apply at checkout.

10 NOTIFICATIONS

10.1 Brief Description

This use case covers the in-app notification system for the ZPIN platform. It includes sending order confirmation notifications to customers when their order is placed, displaying unread notification badge counts, allowing customers to manage their notification preferences, and delivering new-order notifications to sellers when their products are purchased.

10.2 Actors

- Customer — receives order notifications and manages notification preferences
- Seller — receives new order notifications for their products
- System — generates, stores, and delivers notifications; tracks read and unread status

10.3 Flow of Events

10.3.1 Basic Flow

10.3.1.1 Customer Order Confirmation Notification

1) The customer successfully places an order and the payment is verified.

2) The system writes a payment_success notification to the notifications table for the customer's user ID.

3) The customer navigates to the Notifications screen. The system displays the payment success notification with the order number and amount.

4) A badge count is displayed on the relevant tab or icon indicating the number of unread notifications.

5) The customer taps the notification and is navigated to the relevant Order Detail screen.

10.3.1.2 Manage Notification Preferences

1) The customer navigates to the profile settings screen.

2) The system displays toggles for Push Notifications, Email Notifications, and Order Updates.

3) The customer updates their preferences and saves.

4) The system sends a PUT request to the profile details endpoint with the updated notification preference flags.

5) The updated settings are reflected immediately in the UI.

10.3.1.3 Seller New Order Notification

1) A customer places an order containing products from a seller's account.

2) The system creates a new_order notification in the notifications table with the full order details and the decrypted shipping address.

3) The seller fetches notifications from the notifications endpoint and sees unread new_order notifications.

4) Fetched notifications are automatically marked as read.

5) The seller can check the unread count endpoint to see the number of unread new_order notifications.

10.3.2 Alternative Flows

10.3.2.1 Order Updates Disabled by Customer

If the customer has disabled Order Updates in their notification preferences, the system does not send order status change notifications to that customer.

10.3.2.2 Multi-Seller Order Notifications

If an order contains products from multiple sellers, the system creates a separate new_order notification for each seller rather than a single shared notification.

10.3.2.3 No Unread Notifications

If the customer has no unread notifications, the badge count is either not displayed or shows zero, and the Notifications screen shows an appropriate empty state.

10.4 Special Requirements

- Notifications must be stored in a notifications table with a user ID reference, notification type, content, and read status.
- Notifications must be automatically marked as read when they are fetched by the recipient.
- For Razorpay payments, the payment_success notification is sent only after the backend verifies the payment signature.
- Notification preferences are persisted per user and stored via the profile details endpoint.
- Multi-seller orders must generate individual notifications per seller, not a single shared notification.

10.5 Preconditions

- The customer or seller must be registered and authenticated in the system.
- For order confirmation notifications, the order must have been successfully placed and (where applicable) the payment verified.

10.6 Postconditions

- After an order confirmation notification is generated, it appears in the customer's Notifications screen with the correct order details.
- After the customer updates notification preferences, the changes are immediately effective and subsequent notifications respect the updated settings.
- After a seller fetches their notifications, all returned notifications are marked as read.